ASP.NET Programming Study Guide

Prepared by: Aditya Bapusaheb Chaudhari, Full Stack Engineer

Contact Details
LinkedIn: https://www.linkedin.com/in/aditya-chaudhari0301/
GitHub: https://github.com/chaudhari-aditya03
Email: aditya.chaudhari.dev@gmail.com

CS-564-MJ | F.Y. M.Sc. Computer Science | Semester II

Complete Exam Preparation Study Material

Based on Previous Year Question Papers

Exam Coverage: Analysis of QP [6255]-206, [6346]-2006, [6486]-206 | 2023 Pattern | Max Marks: 35

Document Overview & Utility

This Study Guide consists of 9 Structured Units:

- Covers core topics: .NET Framework (CLR, CTS), Web Forms, Page Life Cycle, Server Controls (GridView, FormView), Validation Controls, ADO.NET, MVC, Master Pages, Caching, and Error Handling.

Benefits & Exam Readiness:

- **High-Impact Focus:** The content is meticulously designed by analyzing **4-5 previous question papers**, ensuring focus on the most frequently tested and high-priority concepts.
- **Clarity and Structure:** All technical syntax, examples, and model answers are presented in a clean, simplified, and numbered format, ideal for last-minute revision.
- **Ready for Distribution:** This guide provides a complete, professionally formatted resource designed to ensure every student achieves maximum exam preparedness.
- **Stay Connected:** Students are encouraged to connect with the author, **Aditya Chaudhari**, for updates and networking via My LinkedIn Profile.

UNIT 1: Introduction to ASP.NET & .NET Framework

1.1 What is ASP?

ASP stands for Active Server Pages. It is a server-side scripting technology developed by Microsoft that enables developers to create dynamic, interactive web pages. ASP code runs on the server and the result (HTML) is sent to the client browser. Classic ASP uses VBScript or JScript and embeds code directly in HTML pages.

1.2 What is ASP.NET?

ASP.NET is a web application framework developed and maintained by Microsoft. It is the successor to classic ASP and is built on the .NET Framework. ASP.NET allows developers to build dynamic web applications, services, and websites using languages like C# and VB.NET.

Purpose / Key Features of ASP.NET Framework:

- Server-side web technology — code runs on the web server
- Supports multiple programming languages (C#, VB.NET)
- Event-driven programming model similar to Windows Forms
- Built-in state management (ViewState, Session, Application, Cookies)
- Rich set of server controls (TextBox, Button, GridView, etc.)
- Integrated validation controls
- ADO.NET for database connectivity
- Support for caching, security, and deployment
- Supports MVC (Model-View-Controller) architectural pattern

1.3 CLR – Common Language Runtime

CLR (Common Language Runtime) is the execution engine of the .NET Framework. It is a virtual machine component that manages the execution of .NET programs.

How CLR Works in ASP.NET:

Step	Process
1. Source Code	Developer writes code in C# or VB.NET
2. Compilation	Compiler converts source code to Intermediate Language (IL / MSIL)
3. JIT Compilation	CLR's Just-In-Time compiler converts IL to native machine code at runtime
4. Execution	Native code is executed by the CPU
5. Memory Mgmt	CLR's Garbage Collector automatically manages memory allocation/deallocation

Responsibilities of CLR:

- Memory Management (Garbage Collection)
- Exception Handling

- Security Management (Code Access Security)
- Thread Management
- Type Safety Verification
- JIT Compilation of IL code

1.4 CTS – Common Type System

CTS (Common Type System) is a standard that defines how types (data types) are declared, used, and managed in the .NET runtime. It ensures that objects written in different .NET languages can interact with each other.

Key Points of CTS:

- Defines a rich set of built-in types: integers, strings, booleans, etc.
- All .NET languages share the same base types through CTS
- Enables cross-language inheritance and integration
- Types are divided into: Value Types (int, float, struct) and Reference Types (class, string, array)
- Ensures type safety across all .NET languages

1.5 Namespace in .NET

A namespace is a container that organizes related classes, interfaces, structs, and other types. It prevents name conflicts between different libraries.

- Example: System.Web contains ASP.NET web-related classes
- System.Data contains ADO.NET database classes
- Declared using the 'namespace' keyword in C#
- Used with 'using' directive to import: using System.Web.UI;

1.6 ASP.NET Architecture

Layer	Description
Browser (Client)	Sends HTTP requests, receives HTML/CSS/JS responses
Web Server (IIS)	Internet Information Services — receives requests, routes to ASP.NET runtime
ASP.NET Runtime	HTTP Handler, HTTP Modules, Page Framework process the request
.NET Framework	CLR, CTS, BCL — execute the compiled code
Data Layer	ADO.NET connects to SQL Server or other databases

Exam Tip: ASP.NET Architecture question appears in Q4 of all 3 papers. Draw a layered diagram: Browser → IIS → ASP.NET Runtime → CLR → Database.

UNIT 2: Web Forms, Page Life Cycle & State Management

2.1 Web Forms – Base Class

All Web Forms in ASP.NET are inherited from the System.Web.UI.Page class. This base class provides all the properties, methods, and events needed for a web page to function.

- The Page class inherits from: System.Web.UI.TemplateControl → System.Web.UI.Control → System.Object
- Every .aspx page compiles into a class that inherits from Page
- Provides access to: Request, Response, Session, Application, Server objects

2.2 Creating a Webpage in ASP.NET

Steps to Create a Web Page:

1. Open Visual Studio → File → New → Website (or Project)
2. Select ASP.NET Web Application template
3. Right-click on project → Add → New Item → Web Form
4. Design the page using the .aspx file (HTML + server controls)
5. Write logic in the code-behind file (.aspx.cs for C#)
6. Run using F5 or Ctrl+F5 (Start without debugging)

A web form has two parts:

- .aspx file — the presentation layer (HTML + server controls)
- .aspx.cs file — the code-behind (C# logic)

2.3 Page Life Cycle Events

The ASP.NET Page Life Cycle defines the sequence of events that occur when a page is requested. Understanding this is critical for proper coding.

Order	Event	Description
1	PreInit	First event; create/recreate dynamic controls, set master page/theme
2	Init	Initialize page and controls; ViewState not yet available
3	InitComplete	Initialization complete
4	PreLoad	Before ViewState and postback data is loaded
5	Load	Page.IsPostBack checked here; most code logic written here (Page_Load)
6	Control Events	Postback events like Button_Click are fired
7	LoadComplete	All load events complete
8	PreRender	Last chance to make changes before rendering
9	SaveStateComplete	ViewState saved

Order	Event	Description
10	Render	HTML generated and written to response
11	Unload	Final cleanup; page objects removed from memory

Exam Tip: List at least 6-7 life cycle events in order. Page_Load is the most important. The question appears in Q3 of all 3 papers.

2.4 State Management in ASP.NET

HTTP is a stateless protocol — each request is independent. ASP.NET provides multiple mechanisms to maintain state (data) across requests.

2.4.1 ViewState

ViewState is a client-side state management mechanism used to preserve control values between postbacks on the same page.

- Stored as a hidden field (__VIEWSTATE) in the HTML page
- Data is serialized (Base64 encoded) and stored in the page itself
- Available only on the same page — not across multiple pages
- Exists only for the lifetime of the current page (per postback)
- Items exist as long as the user stays on that page
- Usage: ViewState["key"] = value; var x = ViewState["key"];
- Where stored after postback: In the page's hidden field, sent back to server with each request

2.4.2 Session State

Session State stores user-specific data on the server for the duration of a user's session (visit to the website).

- Stored on the SERVER (in-memory by default, or SQL Server, or State Server)
- Each user gets a unique Session ID (stored in a cookie or URL)
- Data available across multiple pages for the same user
- Session expires after a timeout (default: 20 minutes of inactivity)
- Usage: Session["username"] = "John"; string name = Session["username"].ToString();
- More secure than ViewState — data stays on server

2.4.3 Application State

Application State stores data that is shared across ALL users and ALL sessions of a web application.

- Stored in server memory — accessible to all users
- Persists as long as the application is running (until IIS restarts)
- Usage: Application["visits"] = count; (use Application.Lock() and Application.UnLock() for thread safety)
- Ideal for: site-wide counters, global configuration values

Comparison Table – State Management

Feature	ViewState	Session State	Application State
Storage Location	Client (Hidden Field)	Server	Server
Scope	Single Page	Single User (all pages)	All Users (all pages)
Lifetime	Per postback	Session timeout (20 min)	Application lifetime
Data Security	Low (visible in source)	High	High
Performance	Increases page size	Uses server memory	Shared, fast access

UNIT 3: ASP.NET Server Controls

3.1 HTML Controls vs Web Server Controls

Aspect	HTML Controls	Web Server Controls
Definition	Standard HTML elements with runat='server'	ASP.NET specific tags with rich functionality
Syntax	<input type='text' runat='server'>	<asp:TextBox ID='txt1' runat='server'>
Event Support	Limited — client-side only	Full server-side event model
ViewState	Not supported	Supported by default
Properties	Limited HTML attributes	Rich set of .NET properties
Example	HtmlInputText, HtmlButton	TextBox, Button, GridView

3.2 TextBox Control

The TextBox control allows users to enter text input. It is one of the most commonly used controls in ASP.NET.

Key Properties of TextBox:

- ID — Unique identifier for the control
- Text — Gets or sets the text in the TextBox
- TextMode — SingleLine, MultiLine, or Password
- MaxLength — Maximum number of characters allowed
- Enabled — Enables or disables the control (true/false)
- ReadOnly — Makes the control read-only
- Width / Height — Size of the control
- BackColor / ForeColor — Background and text color

Key Events of TextBox:

- TextChanged — Fires when text is changed and the form is posted back
- OnTextChanged — Event handler attribute

3.3 Button Control

The Button control submits a web form to the server. In ASP.NET, the button click triggers server-side event processing.

- Purpose: Submits the page to the server; triggers Click event
- Key Properties: Text, ID, Enabled, CausesValidation, CommandName
- Key Event: Click (e.g., protected void btnSubmit_Click(object sender, EventArgs e) {...})

3.4 Label Control

The Label control displays static or dynamic text on a web page. It is used to show messages, titles, or output to the user.

- Use: To display text that cannot be edited by the user
- Key Property: Text — sets the text content to be displayed
- Unlike HTML , Label provides full server-side control and ViewState support

3.5 CheckBox Control

Properties of CheckBox:

- Checked — Boolean; true if checked, false if unchecked
- Text — Label displayed next to the checkbox
- AutoPostBack — If true, postback occurs when state changes
- Enabled — Enables/disables the control

3.6 Panel Control

A Panel control is a container control that groups other controls together. It acts like a on the server side.

- Used to: Group controls, show/hide multiple controls at once
- Panel.Visible = false hides all controls inside it
- Useful for multi-step forms or conditional display logic
- Properties: Visible, BackColor, BorderStyle, GroupingText, ScrollBars

3.7 Calendar Control

Properties of Calendar Control:

- SelectedDate — Gets or sets the selected date
- SelectionMode — None, Day, DayWeek, DayWeekMonth
- TodaysDate — Gets or sets today's date
- ShowGridLines — Displays grid lines between dates
- FirstDayOfWeek — Sets the first day of the week
- VisibleDate — Controls which month is displayed

3.8 RadioButton List Control

The RadioButtonList control provides a group of radio buttons where only one can be selected at a time.

- Used for: Single-choice selection from a group of options
- Items can be added statically (in .aspx) or dynamically (in code-behind)
- Key Properties: SelectedItem, SelectedValue, SelectedIndex, RepeatDirection
- Example: A form asking 'Gender: Male / Female / Other'

3.9 GridView Control

The GridView control displays data in a tabular (grid) format. It is the most powerful data display control in ASP.NET.

- Displays data from a data source (database, dataset, list)
- Supports paging, sorting, selecting, editing, and deleting rows
- Automatically generates columns based on data
- Key Properties: DataSource, DataKeyNames, AllowPaging, AllowSorting, AutoGenerateColumns
- Key Events: RowCommand, RowEditing, RowUpdating, RowDeleting, PageIndexChanging

3.10 FormView Control

The FormView control displays one record at a time from a data source. Unlike GridView (which shows multiple records), FormView shows a single record with full customization.

- Supports templates: ItemTemplate, EditItemTemplate, InsertItemTemplate
- Supports paging to navigate between records
- Used for master-detail forms

3.11 Event Handling in ASP.NET

ASP.NET uses an event-driven programming model. Controls raise events (like Click, TextChanged) that are handled by methods on the server.

How to Add an Event Handler:

7. Method 1 – In the .aspx file, add the event attribute: OnClick="btnSubmit_Click"
8. Method 2 – Double-click the control in Visual Studio designer (auto-generates handler)
9. Method 3 – In code-behind, wire up manually: btnSubmit.Click += new EventHandler(btnSubmit_Click);

Event Handler Signature:

```
protected void btnSubmit_Click(object sender, EventArgs e) { /* logic here */ }
```

Common events: Click (Button), TextChanged (TextBox), SelectedIndexChanged (DropDownList), RowCommand (GridView), CheckedChanged (CheckBox).

UNIT 4: Validation Controls in ASP.NET

ASP.NET provides built-in validation controls that check user input before processing. They can work on both client-side (JavaScript) and server-side.

4.1 Types of Validation Controls

Validator	Purpose	Example Use
RequiredFieldValidator	Ensures a field is not left empty	Name, Email fields
CompareValidator	Compares two control values or against a constant	Password = Confirm Password
RangeValidator	Checks if value is within a specified range	Age between 18 and 60
RegularExpressionValidator	Validates against a pattern (regex)	Email format, Phone number
CustomValidator	Custom validation logic (client and/or server)	Complex business rules
ValidationSummary	Displays a summary of all validation errors	At form top/bottom

4.2 RequiredFieldValidator

- Prevents form submission if the associated field is empty
- Key Properties: ControlToValidate, ErrorMessage, Text, Display
- Example: <asp:RequiredFieldValidator ControlToValidate='txtName' ErrorMessage='Name is required' runat='server'/>

4.3 RangeValidator

The RangeValidator checks whether the value of an input control is within a specified range.

- Key Properties: ControlToValidate, MinimumValue, MaximumValue, Type, ErrorMessage
- Type can be: Integer, Double, String, Date, Currency
- Example: Validate age is between 18 and 100

```
<asp:RangeValidator ControlToValidate="txtAge" MinimumValue="18"
MaximumValue="100" Type="Integer" ErrorMessage="Age must be 18-100"
runat="server"/>
```

4.4 CompareValidator

Compares the value of one input control to another control or a fixed value.

- Key Properties: ControlToValidate, ControlToCompare (or ValueToCompare), Operator, Type
- Operator values: Equal, NotEqual, GreaterThan, LessThan, GreaterThanEqual, LessThanEqual, DataTypeCheck
- Common use: Confirm password matches password

4.5 RegularExpressionValidator

Validates the input against a regular expression (regex) pattern.

- Key Properties: ControlToValidate, ValidationExpression, ErrorMessage
- Example patterns:
- Email: \w+([-+.']\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*
- Phone (10 digit): ^[0-9]{10}$
- Pincode: ^[0-9]{6}$
- URL: http(s)?://([\w-]+\.)+[\w-]+(/[\w- ./?%&=]*)?

```
<asp:RegularExpressionValidator ControlToValidate="txtEmail"
ValidationExpression="\w+@\w+\.\w+" ErrorMessage="Invalid email"
runat="server"/>
```

4.6 Client-side vs Server-side Validation

Aspect	Client-side Validation	Server-side Validation
Where runs	Browser (JavaScript)	Web server (C# code)
Speed	Faster — no server trip	Slower — requires postback
Security	Less secure — can be bypassed	More secure — cannot be bypassed
User Experience	Immediate feedback	Feedback after form submission
ASP.NET Controls	Validation controls (auto JS)	Page.IsValid check in code-behind
Best Practice	Use for UX improvement	Always use server-side as final check

Best practice: Use BOTH. Client-side for quick UX feedback; server-side as the reliable, secure final check.

4.7 Good Practices for Validation in ASPX Page

- Always place validators near the control they validate
- Use ValidationSummary to show all errors in one place
- Use Page.IsValid check in code-behind before processing
- Set CausesValidation='false' on Cancel buttons
- Use ValidationGroup to group validators for multi-form pages

UNIT 5: ADO.NET and Database Connectivity

5.1 What is ADO.NET?

ADO.NET (ActiveX Data Objects .NET) is Microsoft's data access technology for .NET applications. It provides a set of classes to connect to databases, execute queries, and retrieve/manipulate data.

- Part of the .NET Framework (System.Data namespace)
- Supports disconnected data access model (Dataset)
- Works with SQL Server, Oracle, MySQL, Access, etc.
- Key components: Connection, Command, DataReader, DataAdapter, Dataset

5.2 ADO.NET Architecture / Working

Component	Class (SQL Server)	Purpose
Connection	SqlConnection	Establishes connection to the database
Command	SqlCommand	Executes SQL queries/stored procedures
DataReader	SqlDataReader	Reads data in forward-only, read-only manner (connected)
DataAdapter	SqlDataAdapter	Bridge between Database and Dataset (disconnected)
Dataset	DataSet	In-memory representation of data (disconnected)
DataTable	DataTable	Represents one table in a Dataset

5.3 Connection Object

The Connection object (SqlConnection) establishes and manages the connection to a SQL Server database.

```
SqlConnection con = new SqlConnection("Server=.;Database=MyDB;Integrated
Security=True;");
con.Open();
// execute commands
con.Close();
```

- Connection String contains: Server name, Database name, Authentication
- Always close connection in a finally block or use 'using' statement
- Key Methods: Open(), Close(), Dispose()
- Key Property: ConnectionString, State (Open/Closed)

5.4 Command Object

The Command object (SqlCommand) executes SQL statements (SELECT, INSERT, UPDATE, DELETE) or stored procedures against the database.

- Key Properties: CommandText (SQL query), CommandType (Text/StoredProcedure), Connection
- Key Methods:
- ExecuteReader() — Returns SqlDataReader for SELECT queries
- ExecuteNonQuery() — Executes INSERT, UPDATE, DELETE; returns rows affected
- ExecuteScalar() — Returns a single value (e.g., COUNT, MAX)

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Students", con);
SqlDataReader dr = cmd.ExecuteReader();
while (dr.Read()) {
    Console.WriteLine(dr["Name"]);
}
```

5.5 Dataset

A Dataset is an in-memory, disconnected representation of data. It can hold multiple tables, relationships, and constraints — like a mini database in memory.

- Works in disconnected mode — no active database connection needed once filled
- Contains DataTable objects, each representing a table
- DataRelation can link tables (like foreign keys)
- Filled using SqlDataAdapter.Fill(dataset)
- Changes tracked using DataSet.AcceptChanges() / RejectChanges()
- Used with: GridView, DropDownList, Repeater as DataSource

5.6 Database Connectivity — Full Example

10. Add connection string in Web.config
11. Create SqlConnection using the connection string
12. Open the connection
13. Create SqlCommand with SQL query
14. Use SqlDataAdapter to fill a DataSet OR SqlDataReader to read data
15. Bind data to a control (e.g., GridView.DataSource = ds; GridView.DataBind())
16. Close the connection

UNIT 6: ASP.NET MVC

6.1 What is MVC?

MVC stands for Model-View-Controller. It is a design pattern (architectural pattern) that separates an application into three interconnected components.

Component	Role	Example
Model	Represents data and business logic	Database entities, data access classes
View	UI — displays data to user	.cshtml Razor files, HTML templates
Controller	Handles user input, updates model and view	HomeController.cs, AccountController.cs

6.2 Key Features of ASP.NET MVC

- Clean separation of concerns (Model, View, Controller)
- Full control over HTML — no ViewState or server controls
- RESTful URL routing (e.g., /Products/Details/5)
- Testable architecture — easy unit testing of controllers
- Supports Razor view engine (.cshtml files)
- Better SEO — clean URLs
- Supports Dependency Injection

6.3 Advantages of ASP.NET MVC

- Clean separation of concerns → easier to maintain
- Full control over HTML markup → better front-end flexibility
- Clean, SEO-friendly URLs
- Easier unit testing (controllers are plain C# classes)
- No ViewState → lighter pages, faster performance
- Supports TDD (Test Driven Development)
- Better suited for large teams — parallel development possible

6.4 Disadvantages of ASP.NET MVC

- Steeper learning curve compared to Web Forms
- More code to write for simple scenarios
- No ViewState (can be seen as both advantage and disadvantage)
- Less drag-and-drop development — not beginner-friendly
- Requires understanding of routing, Razor syntax, and more
- More complex project structure

6.5 Web Forms vs MVC – Comparison

Feature	Web Forms	MVC
Architecture	Event-driven, Page-centric	Pattern-based (MVC)
State Management	ViewState (automatic)	No ViewState — manual
URL Structure	File-based (/Default.aspx)	Route-based (/Home/Index)
HTML Control	Limited — server controls generate HTML	Full control over HTML
Testing	Difficult to unit test	Easy to unit test
Learning Curve	Easier for beginners	Steeper
Performance	Heavier (ViewState)	Lighter pages
Best For	RAD, small apps, beginners	Large, complex, testable apps

UNIT 7: Master Pages, Caching & Website Deployment

7.1 Master Page

A Master Page provides a consistent layout template for multiple pages in an ASP.NET web application. It defines a common structure (header, footer, navigation) that content pages inherit.

- File extension: .master
- Uses ContentPlaceHolder controls to define editable regions
- Content pages use: <%@ Page MasterPageFile="~/Site.master" %>
- Content pages fill the ContentPlaceHolder using <asp:Content> tags
- Benefits: Consistent UI, code reuse, easy site-wide changes, reduced duplication
- Example: Header with logo and nav bar defined once → used on 50 pages

7.2 What is Caching?

Caching is the technique of storing frequently used data in memory (or disk) so that future requests can be served faster without re-computing or re-fetching from the database.

Types of Caching in ASP.NET:

Type	Description	How to Use
Output Caching	Caches the entire page output (HTML)	<%@ OutputCache Duration='60' VaryByParam='none' %>
Fragment Caching	Caches a portion of a page (User Control)	OutputCache directive on .ascx user controls
Data Caching	Stores objects (datasets, strings) in memory	Cache["key"] = value; Cache.Insert(...)
Application Caching	Cache object in HttpContext	HttpContext.Current.Cache["data"] = dataset

- Benefits: Improved performance, reduced database load, faster response time
- Cache can be set with expiration time (sliding or absolute)

7.3 Web.config File

The Web.config file is an XML-based configuration file for ASP.NET web applications. It controls application-wide settings.

- Contains: Connection strings, application settings, authentication mode, session config, error pages, custom HTTP modules
- Located at the root of the web application
- Processed by IIS and ASP.NET runtime before the application starts
- No recompilation needed when Web.config changes — changes take effect immediately

7.4 Deploying a Website on IIS

IIS (Internet Information Services) is Microsoft's web server for hosting ASP.NET applications on Windows.

Steps to Deploy on IIS:

17. Build the ASP.NET project in Release mode in Visual Studio (Build → Publish)
18. Choose publish target: File System, IIS, FTP, or Azure
19. Copy published files to the IIS server folder (e.g., C:\inetpub\wwwroot\MyApp)
20. Open IIS Manager (inetmgr)
21. Right-click Sites → Add Website → Set Site name, Physical path, Port
22. Set Application Pool: .NET CLR Version (v4.0), Managed Pipeline Mode (Integrated)
23. Grant folder permissions to IIS_IUSRS account
24. Update Web.config connection strings for production database
25. Test the website by accessing http://localhost or the server IP

7.5 How to Publish ASP.NET Website (Visual Studio)

26. Right-click the project in Solution Explorer → Publish
27. Select Target: Folder, IIS, FTP, Azure, etc.
28. Configure settings (connection string, configuration: Release)
29. Click Publish — Visual Studio compiles and creates the deployment package
30. Copy the output folder contents to the server

UNIT 8: Error Handling, Tracing & Unit Testing

8.1 Exception Handling in ASP.NET

Exception handling is the process of responding to and recovering from runtime errors (exceptions) gracefully, without crashing the application.

Methods of Exception Handling:

31. Try-Catch-Finally Block (Page-level)

```
try {
    // code that may throw exception
    int x = int.Parse(txtAge.Text);
} catch (FormatException ex) {
    lblError.Text = "Invalid input: " + ex.Message;
} catch (Exception ex) {
    lblError.Text = "Error: " + ex.Message;
} finally {
    con.Close(); // always runs
}
```

32. Page_Error Event — Handles unhandled exceptions at page level
33. Application_Error in Global.asax — Handles unhandled exceptions application-wide
34. Custom Error Pages in Web.config:

```
<customErrors mode="On" defaultRedirect="ErrorPage.aspx">
  <error statusCode="404" redirect="NotFound.aspx"/>
</customErrors>
```

8.2 ASP.NET Error Logging

Error logging involves recording exceptions and errors to a log file or database for later analysis.

- Can use: Windows Event Log, Text File, Database, ELMAH (third-party library)
- Log information typically includes: exception type, message, stack trace, user info, timestamp
- Best practice: Never show stack traces to end users; log them server-side

8.3 Page Level Tracing

Page Level Tracing is an ASP.NET debugging feature that displays detailed information about a page's execution — such as control tree, session state, headers, and timing — at the bottom of the page.

How to Enable Page Level Tracing:

- Add Trace='true' to the @Page directive: <%@ Page Trace='true' Language='C#' %>
- Or enable in Web.config: <trace enabled='true' requestLimit='10' pageOutput='true'/>

Information shown by Tracing:

- Request details (URL, Method, Time)
- Trace Information (custom messages via Trace.Write())
- Control Tree (all controls on the page and their size)
- Session/Application State values
- Request/Response cookies and headers
- Form collection and query string values
- Timing information for each event in page life cycle

8.4 Unit Testing in ASP.NET

Unit Testing is the practice of testing individual units (methods, classes) of code in isolation to verify they work correctly.

- Framework: MSTest, NUnit, or xUnit are used in .NET
- Visual Studio has built-in Test Explorer for running unit tests
- ASP.NET MVC is easier to unit test (controllers are plain classes)
- ASP.NET Web Forms is harder to unit test due to tight coupling with HttpContext

Unit Test Example Structure:

```
[TestClass]
public class CalculatorTests {
    [TestMethod]
    public void Add_TwoNumbers_ReturnsSum() {
        // Arrange
        var calc = new Calculator();
        // Act
        int result = calc.Add(3, 4);
        // Assert
        Assert.AreEqual(7, result);
    }
}
```

UNIT 9: Important Questions & Model Answers

9.1 One-Mark Questions (Q1 Type — 8 Marks)

Question	Answer (1 mark)
Base class for all Web Forms	System.Web.UI.Page
How long do ViewState items exist?	For the duration of the current page (one postback to the next postback on the same page)
How to add an event handler?	Double-click control in designer, OR add OnClick='MethodName' in .aspx, OR += new EventHandler(method) in code
Two properties of TextBox	Text and TextMode (or any two: MaxLength, ReadOnly, Enabled, Width, BackColor)
What is CTS?	Common Type System — defines how data types are declared and used across all .NET languages
What is ASP?	Active Server Pages — Microsoft's server-side scripting technology for dynamic web pages
What is Panel?	A container server control that groups other controls; used to show/hide multiple controls at once
Define Session State	Server-side state management storing user-specific data across multiple pages for the duration of a user's session
Use of Field/Required validator	Ensures a required input field is not left empty before form submission
Define GridView Control	A data-bound control that displays data in a tabular grid format with support for paging, sorting, editing
Define CLR	Common Language Runtime — the execution engine of .NET that manages memory, security, execution, and JIT compilation
Define Private Assembly	An assembly used by only one application; stored in the application's own directory (not in GAC)
Two events of TextBox	TextChanged and OnTextChanged (auto-postback events)
What is a namespace?	A container that organizes related classes and types to avoid name conflicts in .NET
Two properties of CheckBox	Checked (boolean state) and Text (label displayed next to it)
Define View State	Client-side state management using a hidden form field (__VIEWSTATE) to persist control values across postbacks
Use of Label control	Displays static or dynamic text on a web page; non-editable output to the user
Define Application State	Server-side state shared across ALL users and ALL sessions; persists for the application lifetime

Question	Answer (1 mark)
Purpose of ASP.NET framework	To build dynamic, data-driven web applications and services using .NET languages like C# and VB.NET
Define validation control	A server control that validates user input before the page is processed on the server
Define Master Page	A template file (.master) that provides a consistent layout for multiple content pages in an ASP.NET application
Purpose of Web.config	XML configuration file that stores application-wide settings: connection strings, auth mode, session, error pages
What is error handling?	Mechanism to detect, catch, and respond to runtime exceptions gracefully to prevent application crashes
Define website deployment	The process of publishing an ASP.NET web application to a web server (like IIS) for production use
Purpose of Button control	Submits a web form to the server when clicked, triggering the Click server-side event
What is Application State?	Shared, server-side storage accessible by all users — used for global data like visitor counters

9.2 Four-Mark Questions (Q2 & Q3 Type)

Q: Explain Exception Handling

1. ASP.NET uses several ways to handle errors: try-catch-finally blocks, Page_Error events, Application_Error in Global.asax, and custom error pages in Web.config.
2. The 'try' block holds the code that might cause an error.
3. 'Catch' blocks deal with specific types of errors when they happen.
4. The 'finally' block always runs to perform cleanup tasks.
5. The Web.config file uses 'customErrors' to send users to friendly pages instead of showing technical error details.

Q: What is caching? / What is caching in ASP.NET?

1. Caching saves commonly used data in fast memory so the system doesn't have to recreate it or fetch it from the database every time.
2. Output Caching saves the entire HTML of a page using the 'OutputCache' directive.
3. Fragment Caching saves specific parts of a page, like User Controls.
4. Data Caching uses the 'Cache' object to store specific datasets or objects with set expiration times.
5. This results in faster website speeds and less work for the server and database.

Q: Write a short note on Master Page

1. A Master Page (.master file) acts as a layout template to keep a website looking consistent.
2. It uses 'ContentPlaceHolder' controls to mark the areas where content pages can add their own information.
3. Content pages connect to it using the 'MasterPageFile' attribute.
4. The main benefits are a uniform look, reusable code, and the ability to update headers or footers across the whole site in one place.

Q: Explain the difference between ViewState and SessionState

ViewState	SessionState
Stored on client (hidden form field)	Stored on server
Available on same page only	Available across all pages for one user
Exists per postback	Exists until session timeout (20 min default)
Less secure (visible in page source)	More secure
Increases page size	Uses server memory

9.3 Eight-Mark Questions (Q4 Type)

Q: Explain ADO.NET Working

1. ADO.NET uses two models: Connected and Disconnected.
2. In the Connected model, 'SqlConnection' opens the link, 'SqlCommand' runs the query, and 'SqlDataReader' reads data one row at a time.
3. In the Disconnected model, 'SqlDataAdapter' fills an in-memory 'DataSet' and then closes the database connection.
4. You can save changes back to the database later using 'DataAdapter.Update()'.
5. It is best practice to keep connection strings in Web.config and ensure connections are closed in 'finally' blocks.

Q: Explain ASP.NET Architecture

1. ASP.NET Architecture works in several layers.
2. The Client Layer (browser) sends a request, which the IIS Web Server receives and passes to the ASP.NET runtime.
3. The HTTP Pipeline uses modules and handlers to process this request.
4. The Page Framework (Web Forms or MVC) handles events and builds a response.
5. The CLR manages the actual code execution, memory, and errors.
6. The Data Layer uses ADO.NET to talk to databases like SQL Server.
7. Finally, the server sends the resulting HTML back to the browser.

IMPORTANT QUESTIONS LIST (Exam Ready)

Study Strategy: These questions appeared across 3 previous papers. Questions marked *** appeared in multiple papers — highest priority!

Q1 Type — One-Mark (Solve any 8 of 10)

#	Question	Priority
1	Define CLR / How does CLR work in ASP.NET?	***
2	What is CTS (Common Type System)?	***
3	Define Session State	***
4	Define View State / How long do ViewState items exist?	***
5	Define Application State	***
6	What is ASP / What is ASP.NET?	***
7	Define GridView Control	***
8	What is a namespace?	**
9	What is Panel?	**
10	Two properties of TextBox / CheckBox	**
11	How to add an event handler?	**
12	What is the use of Label/RequiredField validator?	**
13	What is MVC?	**
14	Define Master Page in ASP.NET	**
15	What is Web.config? / Define website deployment	*
16	Base class of all Web Forms	*
17	Define Private Assembly	*
18	Two events of TextBox	*
19	What is the purpose of Button control?	*
20	Define a validation control	*

Q2 Type — Short Answer (Attempt any 4 of 5)

#	Question	Priority
1	Explain RangeValidator / CompareValidator	***
2	Write short notes on Event Handling	***
3	What is ADO.NET?	***
4	What are the different validators in ASP.NET?	***

#	Question	Priority
5	What is Dataset?	***
6	Explain FormView control	**
7	List properties of Calendar control	**
8	Explain ASP.NET error logging	**
9	Difference between ViewState and SessionState	**
10	Key features of ASP.NET MVC	**
11	How to create a webpage in ASP.NET?	**
12	What is error handling?	**
13	RadioButtonList control with example	*

Q3 Type — Medium Answer (Attempt any 4 of 5)

#	Question	Priority
1	List the events in the page life cycle	***
2	What is caching in ASP.NET?	***
3	Write a short note on Master Page	***
4	Explain Exception Handling	***
5	Explain Connection and Command Object	***
6	Types of validation controls in ASP.NET	**
7	Good practices to implement validations in ASPX page	**
8	Where is ViewState stored after postback?	**
9	How does CLR work in ASP.NET?	**
10	Explain Unit Testing	**

Q4 Type — Long Answer (Attempt any 2 of 3)

#	Question	Priority
1	Explain ADO.NET working (with diagram)	***
2	Explain ASP.NET Architecture	***
3	Steps to deploy website on IIS / How to publish ASP.NET website	***
4	Advantages of ASP.NET MVC	***
5	Disadvantages of ASP.NET MVC	**
6	Compare Client-side and Server-side validation	**

Q5 Type — 3-Mark Short (Attempt any 1 of 2)

#	Question	Priority
1	What is Page Level Tracing? How to enable it?	***
2	Explain Regular Expression Validator with example	***
3	Distinguish between Web Forms and MVC	***
4	Explain Database Connectivity	**
5	Difference between HTML controls and Web Server controls	**

Final Exam Strategy: *** = appeared in 2-3 papers (MUST study) | ** = appeared in 1 paper (important) | * = supporting knowledge. Focus on: ADO.NET, Page Life Cycle, Validators, State Management, MVC, Master Pages, Architecture, Caching, IIS Deployment. All Q4 and Q5 answers need 5-10 lines minimum.